
ROB-GY 6333 Course Project

Pratyaksh Prabhav Rao Long Quang Rahul Reghunath Jotheesh Kummathi

Siddhant Baroth Sourabh Misal Tarunkumar Palanivelan

Team Doorbusters

Abstract

Autonomous swarm robotics, particularly involving heterogeneous agents like unmanned ground vehicles (UGVs) and aerial vehicles (UAVs), presents promising capabilities for high-risk missions such as room clearing, reconnaissance, and search-and-rescue. However, coordinating such swarms in confined environments—especially where only a limited number of agents can traverse narrow passages simultaneously—remains a significant challenge. This project addresses the problem of decentralized, collision-free navigation through such spatial bottlenecks. We propose a three-layered framework comprising (i) decentralized goal assignment using a distributed graph neural network (DGNN-GA), (ii) swarm coordination to maintain connectivity and avoid congestion, and (iii) local control policies for smooth and safe motion. The DGNN-GA architecture leverages a graph-based representation of robots and goals, performing iterative message passing to compute optimal goal assignments while respecting communication constraints and spatial limitations. Our approach enables adaptive, resilient coordination within heterogeneous swarms, ensuring mission success even in adversarial, cluttered environments.

1 Introduction

Autonomous swarm robotics has emerged as an emerging technology for military and security operations, particularly in tasks such as room clearing, reconnaissance, and search-and-rescue missions [1, 2]. In these high-stakes environments, deploying a heterogeneous swarm—comprising unmanned ground vehicles (UGVs) and unmanned aerial vehicles (UAVs)—offers significant advantages in terms of speed, adaptability, and resilience [3]. However, operating a swarm in confined and cluttered spaces introduces unique challenges: agents must intelligently coordinate their motion, dynamically adjust formations, and navigate narrow passageways without collision [4]. Successful execution demands not only efficient task planning and goal assignment but also real-time cooperation among agents with diverse capabilities.

This project focuses on the problem of autonomously navigating a heterogeneous swarm through a narrow doo, where only a limited subset of agents can pass through critical bottlenecks at any given time. We propose a methodology that decomposes the mission into three key layers: distributed goal assignment [5], decentralized swarm management, and local control policy. By designing intelligent swarm coordination strategies that adaptively manage spatial constraints, our framework enables collision-free traversal and subsequent reformation of the swarm. This capability is crucial for applications requiring stealthy, coordinated entry into hostile environments, ensuring operational success even in the presence of spatial bottlenecks and adversarial conditions.

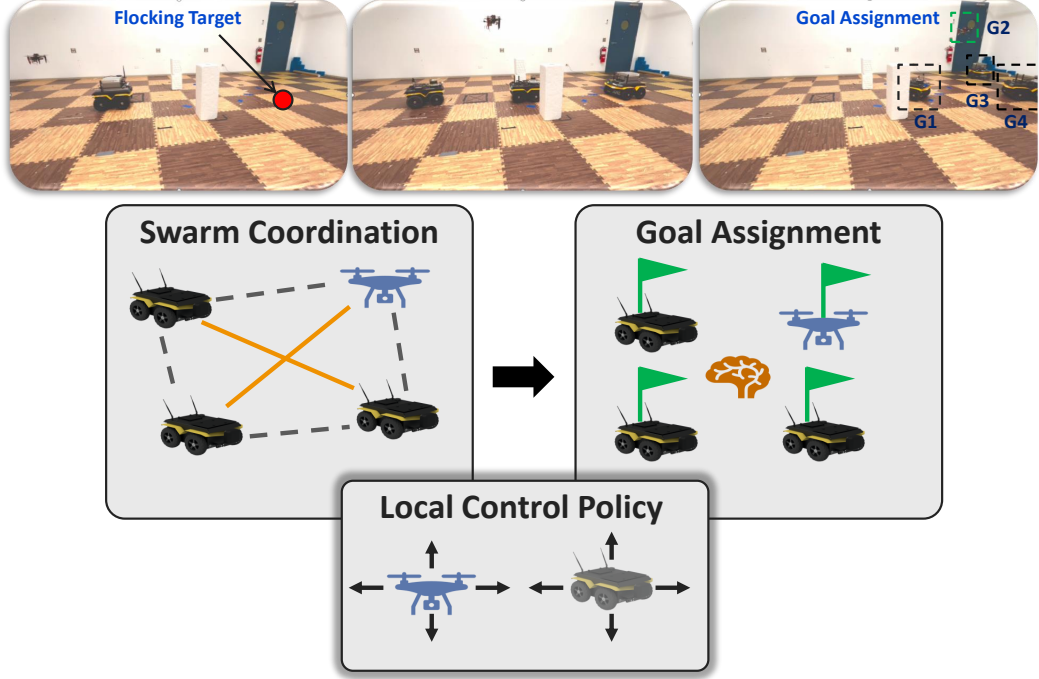


Figure 1: Proposed methodology

2 Problem Formulation

We consider a heterogeneous swarm composed of unmanned ground vehicles (UGVs) and unmanned aerial vehicles (UAVs), denoted collectively as a set of agents $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$. Each agent a_i belongs to a class $c_i \in \{\text{UGV}, \text{UAV}\}$.

Each agent a_i is associated with a state $\mathbf{x}_i \in \mathbb{R}^{n_x}$, which typically includes its position $\mathbf{p}_i \in \mathbb{R}^3$ and velocity $\mathbf{v}_i \in \mathbb{R}^3$. The environment contains a narrow passage region $\mathcal{P} \subset \mathbb{R}^3$, characterized by spatial constraints that restrict the number of agents passing through simultaneously. Specifically, the passage constraint enforces:

$$|\{a_i \in \mathcal{A} \mid \mathbf{p}_i \in \mathcal{P}\}| \leq 2, \quad (1)$$

where only one UGV and one UAV are permitted inside the passage at a time. Each agent is assigned a target goal $g_i \in \mathbb{R}^3$, and the global mission is to navigate the swarm from their initial positions to their assigned goals while satisfying the following requirements:

1. **Goal Achievement:** Each agent must reach its assigned goal location.
2. **Collision Avoidance:** Agents must avoid collisions with each other and obstacles.
3. **Swarm Connectivity:** Agents should maintain network connectivity through local communication.
4. **Passage Constraints:** No more than one UGV and one UAV may occupy the passage simultaneously.

To solve this problem in a scalable, decentralized manner, we decompose it into three layers: (i) decentralized goal assignment, (ii) swarm coordination, and (iii) local control policy, as discussed in the following sections (Refer figure 1).

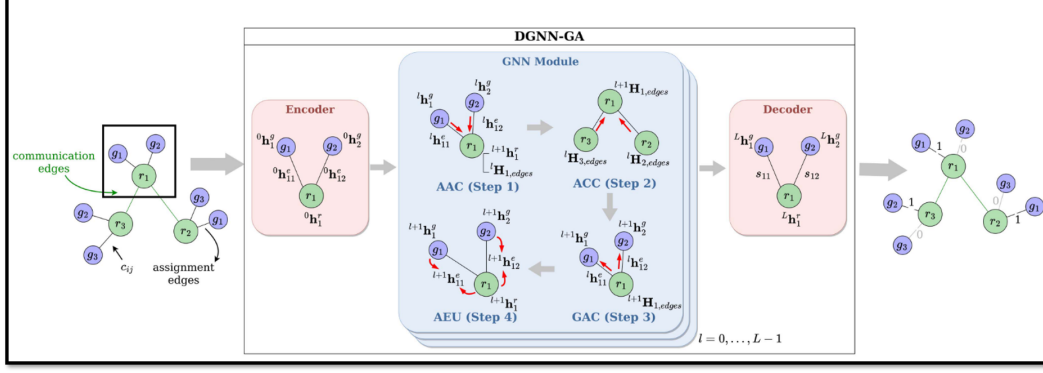


Figure 2: Overview of the DGNN-GA architecture for decentralized goal assignment proposed in [5]

2.1 Goal Assignment via Graph Neural Networks

We adopt a decentralized Graph Neural Network-based approach for goal assignment, based on the DGNN-GA framework [5]. The goal assignment process is structured into three components: *Encoder*, *GNN Module*, and *Decoder*, as illustrated in Fig. 2.

2.1.1 Graph Construction

We model the heterogeneous swarm using two types of nodes:

- Robot nodes $r_i \in \mathcal{R}$
- Goal nodes $g_j \in \mathcal{G}$

Edges are constructed as follows:

- **Communication edges** $(r_i, r_j) \in \mathcal{E}_c$: represent local communication between robots.
- **Assignment edges** $(r_i, g_j) \in \mathcal{E}_a$: represent the potential assignment of robot r_i to goal g_j .

Each node is initialized with a feature vector:

$$\mathbf{h}_i^r(0) \text{ for robots, } \mathbf{h}_j^g(0) \text{ for goals.} \quad (2)$$

Each assignment edge (r_i, g_j) is initialized with a feature $\mathbf{h}_{ij}^e(0)$, encoding the relative cost of assigning robot r_i to goal g_j .

2.1.2 Encoder

The encoder processes the initial node and edge features into a latent space:

$$\mathbf{h}_i^{r(0)}, \mathbf{h}_j^{g(0)}, \mathbf{h}_{ij}^{e(0)}. \quad (3)$$

This latent representation captures the communication edges' costs and assignment edges' costs.

2.1.3 GNN Module

The core of the DGNN-GA framework is the GNN module, which iteratively updates the node and edge features through iterative message passing. At each layer l , the following steps are performed:

1) Agent Assignment Convolution (AAC) Each robot aggregates information from assignment edges:

$$\mathbf{m}_i^{a(l)} = \sum_{g_j \in \mathcal{G}_i} \psi_a(\mathbf{h}_i^{r(l)}, \mathbf{h}_j^{g(l)}, \mathbf{h}_{ij}^{e(l)}), \quad (4)$$

where ψ_a is the assignment message function, and $\mathcal{G}_i$ is the set of candidate goals for robot r_i .

1) Agent Communication Convolution (ACC) Each robot aggregates information from neighboring robot nodes via communication edges:

$$\mathbf{m}_i^{(l)} = \sum_{r_j \in \mathcal{N}_c(i)} \psi_c \left(\mathbf{h}_i^{r(l)}, \mathbf{h}_j^{r(l)} \right), \quad (5)$$

where ψ_c is the communication message function.

3) Goal Assignment Convolution (GAC) Each assignment edge updates its feature based on associated robot and goal nodes:

$$\mathbf{h}_{ij}^{e(l+1)} = \phi_e \left(\mathbf{h}_i^{r(l)}, \mathbf{h}_j^{g(l)}, \mathbf{h}_{ij}^{e(l)} \right), \quad (6)$$

where ϕ_e is the edge update function.

4) Assignment Edge Update (AEU) Finally, robot node features are updated based on aggregated messages:

$$\mathbf{h}_i^{r(l+1)} = \phi_r \left(\mathbf{h}_i^{r(l)}, \mathbf{m}_i^{(l)}, \mathbf{m}_i^{a(l)} \right), \quad (7)$$

where ϕ_r is the robot node update function.

These steps are repeated over L layers of message passing to enable rich interactions between robots and goals.

2.1.4 Decoder and Goal Selection

After L iterations, a decoder predicts assignment scores s_{ij} for each assignment edge (r_i, g_j) :

$$s_{ij} = \psi_d \left(\mathbf{h}_i^{r(L)}, \mathbf{h}_j^{g(L)}, \mathbf{h}_{ij}^{e(L)} \right), \quad (8)$$

where ψ_d is the decoding function.

Each robot then selects the goal with the maximum assignment score:

$$g_i^* = \arg \max_{g_j} s_{ij}. \quad (9)$$

This decentralized framework allows each robot to independently determine its goal assignment using only local communication, while achieving globally coordinated behavior.

2.2 Swarm Coordination

Swarm coordination poses a plurality of complexities to navigate. While three of them are platform and model agnostic, the challenge of distributing algorithmic output to a heterogeneous flock is not so much a difficulty in defining calculation procedures for agent poses and velocities than it is entrusting the vehicle stack to translate these states to agents of different types. For example, the grounded nature of UGVs limits the change in altitude as a function of the terrain. However, the altitude of drones is terrain-independent. It is sufficient to infer that an algorithm whose individual agent output is parameterized by the neighbors' localized states and estimations must factor in the Degrees of Freedom assigned per neighbor. Flocking is such an algorithm.

2.2.1 Extended Olfati-Saber Flocking (OSF)

The algorithm proposed by La et al. (2011)[6] presents a decentralized control strategy that enables a group of autonomous agents to exhibit cohesive flocking behavior while being guided by only a small subset of 'informed' agents who know the desired target destination. The control framework is built on three key components: a local interaction force among neighboring agents, a target tracking force for informed agents, and a damping force that stabilizes motion near the target. The overall control law is as follows:

$$\mathbf{u}_i = f_{\alpha_i} + f_{t_i} + f_{\text{damp}_i} \quad (10)$$

Here, $\mathbf{f}_{\alpha i}$ ensures local cohesion and collision avoidance, $\mathbf{f}_{t i}$ attracts informed agents toward the target, and $\mathbf{f}_{\text{damp}_i}$ reduces velocity when agents are near the target to prevent overshoot. The flocking term consists of two components: a gradient-based interaction and a velocity consensus term. The gradient-based interaction can be an attraction or repulsion term. It is computed as follows:

$$f_{\alpha i} = \sum_{j \in \mathcal{N}_i} \phi_{\alpha} (\|q_j - q_i\|_{\sigma}) n_{ij} + a_{ij}(p_j - p_i) \quad (11)$$

Here, sigma-norm $\|\cdot\|_{\sigma}$ is introduced as a smooth substitute for the Euclidean norm. It is characterized by the following expression:

$$\|z\|_{\sigma} = \frac{1}{\varepsilon} (\sqrt{1 + \varepsilon \|z\|^2} - 1) \quad (12)$$

The other components of the attractive or repulsive force computation are:

$$n_{ij} = \frac{q_j - q_i}{\sqrt{1 + \varepsilon \|q_j - q_i\|^2}} \quad (13)$$

The elements $\mathbf{a}_{ij}(\mathbf{q})$ are representative of an adjacency matrix whose values are calculated for every agent pairing (i, j) using a bump function.

$$a_{ij}(q) = \begin{cases} \rho_h \left(\frac{\|q_j - q_i\|_{\sigma}}{r_{\alpha}} \right), & \text{if } j \neq i \\ 0, & \text{if } j = i \end{cases} \quad (14)$$

with the bump function $\rho_h(\mathbf{z})$ defined as:

$$\rho(z) = \begin{cases} 1 & \text{if } 0 \leq z < h \\ \frac{1}{2} \left(1 + \cos \left(\pi \frac{z-h}{1-h} \right) \right) & \text{if } h \leq z < 1 \\ 0 & \text{if } z \geq 1 \end{cases} \quad (15)$$

The action function $\phi_{\alpha}(\cdot)$ that encapsulates the preceding expression vanishes based on the pose spacing between agents, that is, it vanishes for all $(\mathbf{z} \geq \mathbf{r}^{\alpha})$ with $\mathbf{r}^{\alpha} = \|\mathbf{r}\|_{\sigma}$. In the physical sense, it captures the essence of a potential function - a pairwise attractive/repulsive potential:

$$\Psi_{\alpha}(z) = \int_{d^{\alpha}}^z \phi_{\alpha}(s) ds \quad (16)$$

with the action function itself being:

$$\phi_{\alpha}(z) = \rho_h(z/r_{\alpha}) \phi(z - d^{\alpha}) \quad (17)$$

Here $\phi(\mathbf{z})$ is a sigmoidal function defined as $\phi(\mathbf{z}) = 0.5[(\mathbf{a} + \mathbf{b})\sigma_1(\mathbf{z} + \mathbf{c}) + (\mathbf{a}\mathbf{b})]$, here $\sigma_1(\mathbf{z}) = \mathbf{z}/\sqrt{1 + \mathbf{z}^2}$ and the parameters $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ are constrained as $0\mathbf{a} \leq \mathbf{b}$ and $\mathbf{c} = |\mathbf{a} - \mathbf{b}|/\sqrt{1 + \mathbf{z}^2}$ to guarantee $\phi(\mathbf{0}) = \mathbf{0}$. By making $\phi(\mathbf{0}) = \mathbf{0}$, the system avoids pushing when there is no displacement, preserving physical realism and mathematical well-posedness.

For the tracking term $\mathbf{f}_{t i}$, only informed agents apply the navigation force, which depends on their distance to the target.

$$f_{t i} = \begin{cases} -\frac{K_2}{\|q_i - q_t\|} (q_i - q_t) - \frac{K_3}{\|q_i - q_t\|} (p_i - p_t), & \text{if far} \\ -c_1(q_i - q_t) - c_2(p_i - p_t), & \text{if close} \end{cases} \quad (18)$$

This term causes informed agents to pull the swarm towards the target when far, and gently align position and velocity as they get close. Lastly, the damping term $\mathbf{f}_{\text{damp}_i}$ is applied only when an agent is sufficiently close to the target, acting as a braking force:

$$f_{\text{damp}_i} = \begin{cases} -K_{\text{damp}} \cdot p_i, & \text{if } \|q_i - q_t\| < K_4 \cdot r \\ 0, & \text{otherwise} \end{cases} \quad (19)$$

2.2.2 Decentralized Algorithm Implementation

The implementation of the decentralized flocking algorithm was designed with modularity and a clear flow of control computation, allowing it to be both intuitive and language-agnostic. This modular design enabled flexible development, debugging, and tuning across a range of computing environments, each with varying levels of implementation complexity.

Initially, the algorithm was developed and tested in MATLAB, where the focus was on high-level validation of agent behaviors. MATLAB provided a rapid prototyping environment to debug the force computations and visualize the effects of key parameters such as inter-agent spacing, attraction-repulsion balance, and consensus dynamics. Once the parameter tunings in MATLAB reliably produced cohesive, collision-free flocking with stable consensus, these values were carried forward into a ROS 2 Humble implementation.

In ROS 2, the algorithm was integrated into a modular node-based architecture and tested in simulation using Unity3D, which provided a visual and physics-consistent environment to evaluate the interaction between the control algorithm and onboard autonomy stacks.

Table 1: Tunable Parameters Grouped by Function

Function	Parameter	Description
compute_flocking_force	ϵ	Smoothness factor for σ -norm
	D	Desired inter-agent distance
	k_c	Scaling factor to compute neighborhood radius R
	h	Width of transition in bump function ρ
	a, b	Sigmoid potential shaping parameters (attraction and repulsion)
	Derived: R, R_σ, c, d_α	Radius and constants derived from above parameters
compute_navigation_force	K_1	Distance threshold for switching control laws
	K_2	Gain for position attraction when far from target
	K_3	Gain for velocity alignment when far from target
	c_1	Gain for position attraction when close to target
	c_2	Gain for velocity attraction when close to target
compute_damping_force	K_{damp}	Damping gain near the target
	K_4	Distance factor to activate damping

2.3 Local Control Policy

Once goal assignment and swarm coordination are completed, each agent must independently track its assigned trajectory while ensuring smooth, collision-free motion. We implement distinct local control policies for UAVs and UGVs to account for their differing dynamics.

2.3.1 UAV Local Control Policy

The UAVs employ a cascaded control architecture from PX4 [7], operating across multiple layers of abstraction, as shown in Fig. 8.

- **Position Control (50 Hz):** A proportional (P) controller computes the desired velocity $\mathbf{V}_{\text{sp}}$ based on the position error relative to the reference point $\mathbf{X}_{\text{sp}}$.
- **Velocity Control (50 Hz):** A PID controller regulates the desired acceleration $\mathbf{A}_{\text{sp}}$ based on the velocity tracking error.

Algorithm 1 Decentralized Flocking Control with Target Guidance and Damping

Require: Current state (q_i, p_i) of agent i , neighbor states $\{(q_j, p_j)\}_{j \in \mathcal{N}_i}$, target state (q_t, p_t) if informed

Ensure: Control input u_i

```
1: Compute  $\|z\|_\sigma \leftarrow \frac{1}{\epsilon}(\sqrt{1 + \epsilon\|z\|^2} - 1)$ 
2: Compute  $r_\alpha \leftarrow \|R\|_\sigma, d \leftarrow \|d\|_\sigma$ 
3: Initialize  $f_{\alpha_i} = 0$ 

4: for all  $j \in \mathcal{N}_i$  do
5:    $z \leftarrow q_j - q_i$ 
6:    $n_{ij} \leftarrow \frac{z}{\sqrt{1 + \epsilon\|z\|^2}}$ 

7:    $\rho \leftarrow$  bump function:  $\rho(z) = \begin{cases} 1, & \text{if } z < h \\ \frac{1}{2}(1 + \cos(\pi \frac{z-h}{1-h})), & \text{if } h \leq z < 1 \\ 0, & \text{otherwise} \end{cases}$ 

8:    $\phi(z) \leftarrow \frac{1}{2} \left[ (a+b) \frac{z+c}{\sqrt{1+(z+c)^2}} + (a-b) \right]$ 
9:    $\phi_\alpha \leftarrow \rho(\|z\|_\sigma / r_\alpha) \cdot \phi(\|z\|_\sigma - d)$ 
10:   $a_{ij} \leftarrow \rho(\|z\|_\sigma / r_\alpha)$ 
11:   $f_{\alpha_i} \leftarrow f_{\alpha_i} + \phi_\alpha \cdot n_{ij} + a_{ij}(p_j - p_i)$ 
12: end for

13: if agent  $i$  is informed then
14:    $d_i \leftarrow \|q_i - q_t\|, d_0 \leftarrow \|q_i(0) - q_t(0)\|$ 
15:   if  $d_i > K_1 \cdot d_0$  then
16:      $f_{t_i} \leftarrow -\frac{K_2}{d_i}(q_i - q_t) - \frac{K_3}{d_i}(p_i - p_t)$ 
17:   else
18:      $f_{t_i} \leftarrow -c_1(q_i - q_t) - c_2(p_i - p_t)$ 
19:   end if
20: else
21:    $f_{t_i} \leftarrow 0$ 
22: end if

23: if  $\|q_i - q_t\| < K_4 \cdot R$  then
24:    $f_{\text{damp}_i} \leftarrow -K_{\text{damp}} \cdot p_i$ 
25: else
26:    $f_{\text{damp}_i} \leftarrow 0$ 
27: end if

28: return  $u_i = f_{\alpha_i} + f_{t_i} + f_{\text{damp}_i}$ 
```

- **Acceleration to Attitude Mapping (50 Hz):** The desired acceleration and commanded yaw ψ_{sp} are converted into a desired attitude quaternion $\mathbf{q}_{\text{sp}}$.
- **Angle Control (250 Hz):** A proportional (P) controller tracks the attitude by computing desired angular rates Ω_{sp} .
- **Angular Rate Control (1 kHz):** A PID controller stabilizes the body rates by computing the actuator commands δ_{sp} , which are then fed into the motor mixer to produce the final thrust and torque inputs.

This hierarchical approach enables fast, accurate tracking of 3D trajectories while maintaining stability even under external disturbances.

2.3.2 UGV Local Control Policy

For UGVs, we implement a simpler control structure suitable for ground robots moving in planar environments. The control stack consists of:

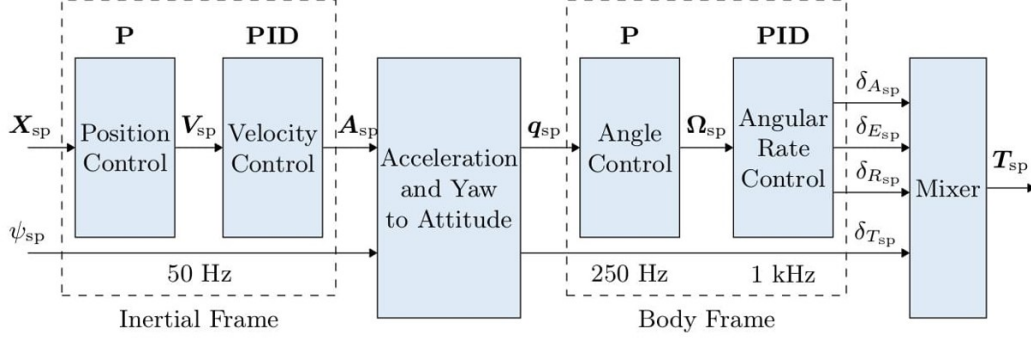


Figure 3: PX4 [7] control stack.

- **Position Controller:** A proportional controller computes a desired velocity command based on the position error relative to the assigned goal.
- **Velocity Controller:** A PID controller regulates the commanded wheel velocities to track the desired translational and rotational speeds.

Since UGVs primarily operate on flat surfaces and are subject to simpler dynamics compared to UAVs, a two-layer position and velocity control structure is sufficient to achieve smooth goal tracking. The controllers are tuned to prioritize stability and responsiveness, ensuring reliable operation even in cluttered, constrained environments.

3 Results

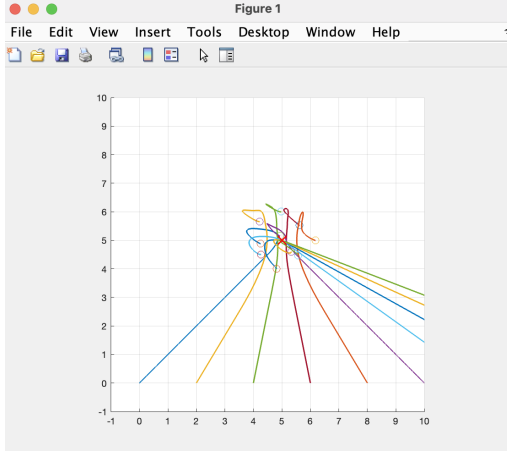
3.0.1 MATLAB Simulations

The setup ensures that the parameters for each drone in the swarm are consistent. All parameters used to calculate the flocking force (attraction and repulsion), the navigation term, and the damping of each agent's pose and velocity, are kept the same for every drone. Furthermore, each agent is set to be informed of the target's pose and velocity. Default values are set for parameters that are used to calculate the sigmoid function and the bump function. Varying them helps tune relative behavior in the neighborhood.

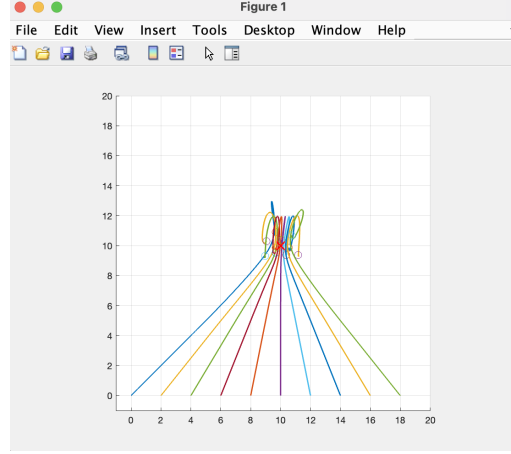
Recommended values of K2 are $K2 > 0$. In order to exercise observing the extremities of behavioral parameters, this value was set to 10.5. Time to target remains the same. The attractive force is far greater and this is observed from the relative distances that agents in the swarm maintain.

K3 determines a similar weight on the pose of the agents to ensure that attractive forces are applied initially. The difference in increasing a weight on the poses versus increasing the weight only on the velocities of the agents is apparent.

K4 determines the impact of damping on the states of the agents as they approach the target. By initializing $K4 = 10.0$, the agents approach the target pose with more damping weight which is observed from the very minimal overshoot of the path they take. There is no rebound.

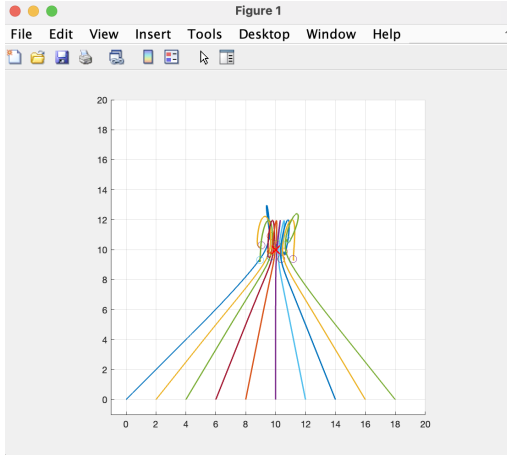


(a) $K2 = 1.5$

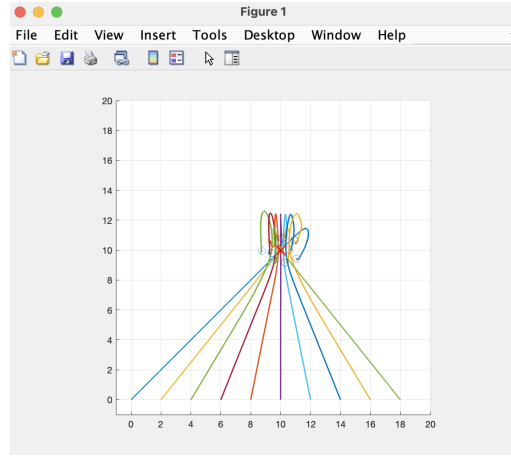


(b) $K2 = 10.5$

Figure 4: Comparison of agent behavior under different $K2$ tuning configurations

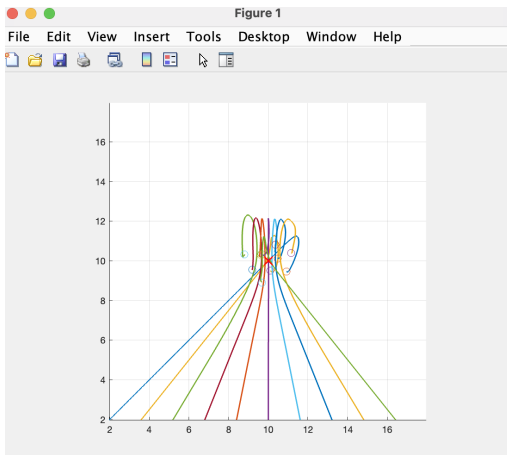


(a) $K2 = 10.5$

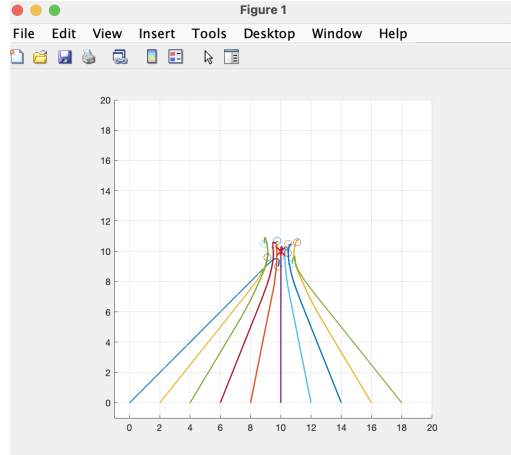


(b) $K3 = 10.0$

Figure 5: Comparison of applying tuning weights on poses and velocities independently



(a) $K4 = 1.5$



(b) $K4 = 10.0$

Figure 6: Comparison of increasing damping weights on trajectory length and overshoot

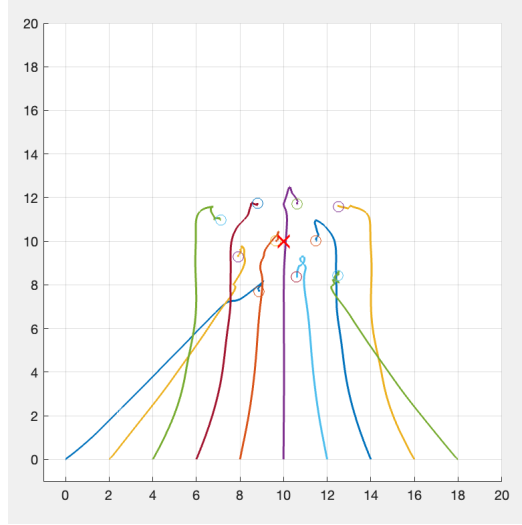


Figure 7: Finalized set of tuning parameters showing desired formation around global target of 10 agents.

In our implementation of the Olfati-Saber Flocking algorithm, we selected a relatively high damping gain $K_{\text{damp}} = 2.0$, along with target tracking gains $K_2 = 1.2$, $K_3 = 1.0$, and a damping activation threshold $K_4 = 10.0$. The choice of a high damping value was motivated by the need to suppress overshoot and oscillatory behavior as informed agents approach the target. While higher damping improves stability and helps agents settle near the goal without excessive velocity, it can also reduce responsiveness if tuned too aggressively.

Thus, we paired this with moderately high values of K_2 and K_3 to maintain a strong attraction towards the target in both position and velocity state space. The damping threshold $K_4 = 1.5$ ensures that the braking force activates early enough — at 1.5 times the neighborhood radius — allowing agents to begin decelerating well before reaching the target. This balance of parameters enables smooth convergence and cohesive swarm behavior, avoiding sharp stops or erratic motion near the goal.

The relative best set of parameters chosen after several combinations exhibit a smooth flock that creates an alpha lattice around the global target.

3.0.2 Unity Simulations



Figure 8: Successful homogeneous UGV flocking of 5 agents in Unity

The implementation of the Olfati-Saber Flocking (OSF) algorithm in Unity, integrated with ROS 2 via real-time messaging and control feedback, demonstrated stable and coherent swarm behavior under moderate conditions. Despite the presence of initial oscillations—particularly forward-backward motions during long-range navigation—the agents consistently exhibited convergence toward the target while maintaining relative cohesion. The differential drive agents responded predictably to the dynamically tuned parameters, and the use of a Unity-based physics environment allowed for detailed evaluation of inter-agent spacing and trajectory smoothness.

Most importantly, while oscillations were observed in early iterations, they were controllable through damping and gain adjustments, especially that of $K_2 > 0$ and $K_3 > 0$ validating the robustness of the control formulation when deployed in a realistic 3-D simulation pipeline.

References

- [1] Manuele Brambilla, Eliseo Ferrante, Mauro Birattari, and Marco Dorigo. Swarm robotics: a review from the swarm engineering perspective. *Swarm Intelligence*, 7:1–41, 2013.
- [2] Andreas Kolling, Katia Sycara, Steve Nunnally, and Michael Lewis. Human swarm interaction: An experimental study of two types of interaction with foraging swarms. *Journal of Human-Robot Interaction*, 2(2), 2013.
- [3] Saumitra M Das, Y Charlie Hu, CS George Lee, and Yung-Hsiang Lu. Mobility-aware ad hoc routing protocols for networking mobile robot teams. *Journal of communications and networks*, 9(3):296–311, 2007.
- [4] Riccardo Falconi, Lorenzo Sabattini, Cristian Secchi, Cesare Fantuzzi, and Claudio Melchiorri. Edge-weighted consensus-based formation control strategy with collision avoidance. *Robotica*, 33(2):332–347, 2015.
- [5] Manohari Goarin and Giuseppe Loianno. Graph neural network for decentralized multi-robot goal assignment. *IEEE Robotics and Automation Letters*, 9(5):4051–4058, 2024.
- [6] Hung Manh La, Ronny Salim Lim, Weihua Sheng, and Heping Chen. Decentralized flocking control with a minority of informed agents. In *Proceedings of the 6th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, pages 1851–1856, Beijing, China, 2011. IEEE.
- [7] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *2015 IEEE international conference on robotics and automation (ICRA)*, pages 6235–6240. IEEE, 2015.